

Witness — Product Requirements Document (Hackathon Build)

Status: Hackathon scope, v1.2 (6 detectors + learning loop + MongoDB + voice) ·

Event length: 36 hours · **Team size:** 4 **Source of truth:** This file. If code, a chat, or an AI assistant disagrees with this PRD, the PRD wins until the team agrees to change it (see §15 Change Process).

0. How to use this document (read first — humans AND AI assistants)

1. Put this file at the **repo root** as **PRD.md**.
2. Every teammate: at the start of every AI coding session (Claude Code, Cursor, ChatGPT, etc.), tell your AI:

"Read **PRD.md** fully before writing any code. Follow §12 (Interface Contracts) exactly and §14 (Rules for AI Assistants). Only work inside the folders owned by my role: **[your role from §6]**."

3. Never change anything in §12 (Interface Contracts) without telling the whole team. Those contracts are what let four people build in parallel.
 4. Requirement IDs (e.g. **FR-C3**) are stable. Reference them in commit messages and PR titles.
-

1. Summary

Witness is a macOS desktop companion — a small animated pet — that appears on the corner of the **Claude desktop app** window, reads the conversation through the macOS Accessibility API, detects hallucinations in the AI's replies, and visibly reacts. The worse the hallucinations get, the more alarmed the pet becomes. Clicking the pet shows a plain-English explanation of what is wrong. Clicking **Fix it** inserts a tailored correction prompt (**Course Correct**) into Claude's message box; the user presses Enter, and the AI corrects itself. At high alert levels the pet also **speaks** (ElevenLabs voice).

Witness learns. Every confirmed hallucination and every correction outcome is stored in **MongoDB Atlas**. Course Correct uses reinforcement learning (a multi-armed bandit) to learn

which correction prompts actually fix the AI, and the detectors use a memory of past confirmed cases to judge new claims better (§7.5).

One-liner for the pitch: *"Witness is a second pair of eyes for every AI you use — it helps the AI fix its own mistakes, and it gets better at fixing them with every hallucination it catches."*

2. Problem

- AI assistants state wrong information with the same confidence as correct information.
 - Users can't fact-check every sentence; they usually find out an answer was wrong only after relying on it.
 - Existing tools (browser fact-check extensions) check one reply at a time inside a browser tab, only flag problems, and don't help the AI recover.
-

3. Goals and Non-Goals

3.1 Goals (hackathon)

ID	Goal	Measured by
G1	Detect hallucinations in Claude desktop app replies in near real time	Verdict shown ≤ 8 s after reply finishes (target ≤ 5 s)
G2	Communicate severity visually and clearly	Pet shows 5 distinct levels driven by a heat score
G3	Explain problems in plain English	Speech bubble lists specific problems + evidence
G4	Help the AI recover	"Fix it" inserts a level-matched correction prompt; next reply re-verified
G5	Prove it works	Evaluation on ~38 trap prompts: report precision and fix rate

ID	Goal	Measured by
G6	Deliver a reliable, memorable 3-minute demo	Demo script (§11) runs end to end 3 times in a row
G7	Show that Witness learns	Learning chart: Course Correct fix rate per round over ≥ 5 seeded evaluation rounds (§17.1), clearly labeled as seeded

3.2 Non-Goals (explicitly out of scope for the hackathon — roadmap only)

- ChatGPT app support (**stretch** only if everything else is done; see §5.3)
- Windows / Linux support
- Browser extension version (only as the **fallback plan**, §13 R1)
- More than one animal in the running app (other animals = pitch-slide mockups)
- Rive animations (use SwiftUI shapes)
- Detectors beyond the six main ones (§7.0): execution checker, contradiction tracker, instruction auditor, action verifier, made-up app/menu steps are **roadmap**
- Code API checks for languages other than Python (JavaScript etc. get package-existence checks only)
- Checking attached files (PDFs uploaded to Claude); the Source Faithfulness Checker covers **pasted text** only
- "Show me" on-screen highlighting of the exact wrong sentence (**roadmap**)
- Auto-sending messages. **Witness never presses Enter. Ever.**
- Accounts, cloud sync, team dashboards
- **Training or fine-tuning model weights.** Witness learns by choosing prompts (bandit), remembering confirmed cases, and tuning thresholds — not by training a neural network (§7.5)
- **Web scraping** for "signs of hallucination". Use public labeled datasets instead (HaluEval, TruthfulQA, FEVER; FR-L6)
- Learning from unconfirmed verdicts (see the confirmed-only rule, FR-L5)
- Storing full conversations in the cloud

4. Users and User Stories

Primary user: a student or professional who uses the Claude desktop app for research, writing, or coding.

ID	As a user, I want...	So that...
US1	the pet to appear only when Claude is in front	it's not cluttering my desktop otherwise
US2	the pet's face to show how trustworthy the conversation is right now	I know when to be skeptical at a glance
US3	to click the pet and read what exactly is wrong, in plain English	I understand the problem without investigating
US4	to see the evidence (source link / lookup result)	I can trust the flag
US5	to click Fix it and get a ready-made correction in the message box	I can get the AI back on track with one Enter press
US6	to tell Witness it's wrong ("I disagree")	false alarms stop bugging me
US7	Witness to never send anything on my behalf	I stay in control

5. Scope

5.1 Hackathon scope vs. full vision

Area	Built at the hackathon	Full vision (roadmap / pitch only)
Apps	Claude desktop app only	Claude, ChatGPT, Claude Code, Windows
Pet	1 animal, drawn in SwiftUI (separate eye, eyebrow, mouth shapes)	4 animals (Owl, Rubber Duck, Fox, Red Panda) animated in Rive

Area	Built at the hackathon	Full vision (roadmap / pitch only)
Levels	5 levels via expression swaps + sweat drop + red flag	Smooth blended animations, signature moves
Detectors	6 main hallucination types (§7.0): Reference Auditor, Claim Verifier, Consistency Probe, Pushback Detector, Source Faithfulness Checker, Code API Checker	10+ detectors, more languages
Course Correct	Diagnosis + level-matched prompts	+ fix-rate learning per template/app
Bubble / Fix it	Speech bubble; Fix it via clipboard paste	+ direct text insert, "Show me" highlighting
Evaluation	~38 trap prompts covering all 6 types; precision + fix rate	Full HaluEval / TruthfulQA / FEVER runs
Learning	Prompt bandit (P0), dataset seeding + learning chart (P0), experience memory via Atlas Vector Search (P1), threshold calibration (P1)	Continuous learning across all users (opt-in)
Storage	MongoDB Atlas for learning data (anonymized); session history local	Team dashboards on Atlas
Voice	ElevenLabs voice at Alarmed/Meltdown and when the bubble opens, cached common lines, mute button (P1)	Per-animal voices

5.2 Default pet for the build

Owl (default; team may swap before hour 2 — see §16 Open Questions). All other animals appear only as mockups in pitch slides.

5.3 Stretch goals (only after all must-haves pass, in this order)

1. Direct AX text insert (before paste fallback)
2. ChatGPT desktop app support
3. Pre-send guardrail tip at level 0
4. Code API checks for JavaScript/TypeScript

5.4 Prize tracks this build targets

- **MLH Best Use of MongoDB Atlas:** learning store + Atlas Vector Search for experience memory (§7.5).
- **MLH Best Use of ElevenLabs:** the pet's voice (§7.6).

6. TEAM MEMBERS AND ROLES ★ (read this section carefully)

We have **4 team members**, and **each person owns exactly one role**. Fill in the names below before the hackathon starts. Each person **owns specific folders** and **specific contracts**. Don't edit another person's folders without asking them.

Team member	Name	Role	Needs a Mac?	Main output
Member 1	----- --	 Role A: Companion Engineer (Swift)	Yes	The pet app on the Claude window (+ plays the pet's voice)
Member 2	----- --	 Role B: Engine Engineer (Python)	No	The server that checks replies and computes heat + the Pushback Detector + MongoDB storage + voice generation
Member 3	----- --	 Role C: Detector Engineer (Python)	No	4 detectors: references, facts, consistency,

Team member	Name	Role	Needs a Mac?	Main output
				code APIs + experience memory
Member 4	----- --	 Role D: Course Correct + Eval + Pitch Lead	No	Correction prompts, the Source Faithfulness Checker, the prompt-learning bandit, accuracy numbers, the pitch

Mac requirement: Only **Role A** requires a Mac. Roles B, C and D can use any OS.

Assignment rule: The teammate without a Mac takes **Member 2 or Member 3**.

Member 1 must have a Mac with Xcode installed.

MEMBER 1 — ROLE A: Companion Engineer (Swift / macOS) — *Mac required*

Name: ----- **Owns folders:** `/companion/**` **Mission:** Everything the user sees and everything that touches the Claude app.

Responsibilities

- Run the **Day-One Test** (`day_one_test.swift`) in hours 0–2 and report pass/fail to the team.
- Detect when the Claude app is frontmost; show/hide the pet (FR-A1).
- Attach the pet window to the Claude window corner and follow it (FR-A2).
- Read the conversation through the Accessibility API; detect new, completed assistant replies and user messages (FR-A3, FR-A4).
- Send messages to the engine over WebSocket per §12 (FR-A5).
- Render the pet: 5 expression levels + Recovered state + sweat drop + red flag (FR-A6, FR-A7).
- Speech bubble UI with buttons: **Fix it**, **Details**, **I disagree**, **Dismiss** (FR-A8).
- Fix it: insert the prompt into Claude's message box via clipboard paste, restoring the user's clipboard (FR-A9).
- Onboarding screen explaining the Accessibility permission (FR-A10).

- Play the pet's voice lines from `voice.play` messages, with a mute button (FR-V3).

Deliverables: `Witness.app` (Xcode project), mock engine mode for solo testing. **Depends on:** Role B's WebSocket server + message contracts (§12). Until then, use `/shared/fixtures/*.json` and the companion's mock mode. **Definition of done:** Demo steps 1–3 of §11 work against the real engine, 3 runs in a row.

■ MEMBER 2 — ROLE B: Engine Engineer (Python / FastAPI) — *any* OS

Name: _____ **Owns folders:** `/engine/app/**` (except `/engine/app/detectors/**`), `/engine/app/detectors/pushback.py`, `/shared/schemas/**` (as schema steward) **Mission:** The brain's backbone — receive messages, orchestrate detectors, keep state, compute heat, push results.

Responsibilities

- FastAPI app with WebSocket endpoint `/ws` and `GET /health` (FR-B1).
- **Schema steward:** own `/shared/schemas/*.json` and the Pydantic models that mirror them (FR-B2). Publish v1 schemas by **hour 2**.
- Claim extraction from assistant replies (FR-B3).
- Risk triage: decide which claims go to which detectors (FR-B4).
- Run detectors in parallel with timeouts; aggregate into one verdict per claim (FR-B5).
- Session ledger in SQLite (FR-B6).
- Heat score + level calculation per §9 (FR-B7).
- Push `verdicts.update`, `heat.update`, `bubble.content` to the companion (FR-B8).
- Debug endpoint `POST /debug/message` so anyone can test without the Mac app (FR-B9).
- **Pushback Detector** (FR-C4), file `engine/app/detectors/pushback.py`. Member 2 owns it because it depends on conversation history the engine already tracks.
- Keep track of **source documents** the user pastes (FR-B11) so the Source Faithfulness Checker can use them.
- **MongoDB Atlas learning store** (FR-L1): connection, collections and indexes per §12.7; write anonymized records.
- **Voice generation** (FR-V1, FR-V2): call ElevenLabs, cache common lines, send `voice.play` to the companion.

Deliverables: Running engine: `uvicorn app.main:app --port 8765`. **Depends on:** Role C detector interface (§12.4) — agree on it by hour 2. **Definition of done:** Posting a fixture reply to `/debug/message` produces correct verdicts, heat, and bubble content within 8 s.

MEMBER 3 — ROLE C: Detector Engineer (Python) — *any OS*

Name: _____ **Owns folders:** `/engine/app/detectors/**` (except `pushback.py` → Member 2 and `source_faithfulness.py` → Member 4), `/engine/tests/detectors/**` **Mission:** Catch invented sources, wrong facts, made-up niche facts and made-up code APIs.

Responsibilities

- **Reference Auditor** (FR-C1) — must-have
- **Claim Verifier** (FR-C2) — must-have
- **Consistency Probe** (FR-C3) — must-have
- **Code API Checker** (FR-C7) — must-have (Python)
- **Experience memory** (FR-L3) — P1, after the Code API Checker: retrieve similar confirmed past cases from Atlas Vector Search and give them to the judge models as examples
- Every detector implements the common interface in §12.4 and never raises uncaught exceptions (returns `status: "error"` instead).
- Unit tests with fixtures for each detector.

Deliverables: `detectors/reference_auditor.py`, `claim_verifier.py`, `consistency_probe.py`, `code_api_checker.py`. **Depends on:** Role B's **Claim** model (§12.3). **Definition of done:** Each detector passes its fixture tests; on the eval set, detectors contribute to precision ≥ 0.80 .

MEMBER 4 — ROLE D: Course Correct + Evaluation + Pitch Lead — *any OS*

Name: _____ **Owns folders:** `/engine/app/course_correct/**`, `/engine/app/detectors/source_faithfulness.py`, `/eval/**`, `/pitch/**` **Mission:** Turn detections into fixes, prove it works with numbers, and win the room.

Responsibilities

- Diagnosis module: drift profile (what went wrong, blast radius, trend, root cause) (FR-D1).
- Prompt builder: 5-part structure, level-matched prompt types, templates per failure type (FR-D2, FR-D3).
- Bubble copy: plain-English headline / problems / pattern text per level (FR-D4).

- Fix-rate tracking: after a correction is inserted, verify the next reply and log fixed/not fixed (FR-D5).
- **Source Faithfulness Checker** (FR-C6), file `engine/app/detectors/source_faithfulness.py`: catches summaries of long pasted documents that add or change facts. Mostly one well-written judge prompt.
- **Prompt-learning bandit** (FR-L2): write 3 prompt variants per failure type; Thompson sampling picks variants; verified fixes are the reward.
- **Dataset seeding + learning chart** (FR-L6, FR-L7): import public dataset examples and run seeded rounds to produce the learning curve for the pitch.
- Threshold calibration from "I disagree" feedback (FR-L4, P1).
- Evaluation: build ~38 trap prompts covering all 6 types, run them, compute precision + fix rate (FR-D6).
- Pitch deck (incl. mockups of the other 3 animals), demo script rehearsal, backup demo video (FR-D7).

Deliverables: `course_correct/` module, `source_faithfulness.py`, `eval/results.md`, pitch deck, backup video. **Depends on:** Role B verdict/ledger models; Role C detector outputs. **Definition of done:** Level 1–4 prompts generated correctly for all fixture scenarios; eval numbers in the deck.

6.1 Individual hour-by-hour checklists

Each member follows their own column. Tick items off in your copy; blockers go in team chat immediately.

Member 1 — Companion (Swift)

- ☐ **Before the event:** install Xcode, free 40+ GB of disk, run `day_one_test.swift` once at home
- ☐ **h0–2:** run the Day-One Test on the Claude app; post the results (can read? can paste? bundle ID?) in team chat
- ☐ **h2–6:** Xcode project; floating pet panel that shows/hides with the Claude app (FR-A1, FR-A2)
- ☐ **h6–10:** AX reader sends `message.new` for completed replies (FR-A3–A5); mock mode from fixtures (FR-A11)
- ☐ **h10 Integration 1:** real Claude reply reaches the engine
- ☐ **h10–18:** SwiftUI pet face: 5 levels + accessories (FR-A6, FR-A7)
- ☐ **h18 Integration 2:** fake citation turns the pet Alarmed
- ☐ **h18–24:** speech bubble + Details panel + I disagree (FR-A8)
- ☐ **h24–30:** Fix it paste into Claude's message box (FR-A9); onboarding sheet (FR-A10); voice playback + mute (FR-V3)

- ☐ **h30–36:** polish animations, fix bugs, rehearse demo

Member 2 — Engine (Python)

- ☐ **Before the event:** Python 3.11, API keys ready (Anthropic, Tavily, ElevenLabs), FastAPI hello-world running; create a free MongoDB Atlas cluster and test a connection
- ☐ **h0–2:** repo skeleton + `.env.example`; publish §12 schemas v1 and Pydantic models (FR-B2)
- ☐ **h2–4:** publish `/shared/fixtures` incl. 7 scenarios
- ☐ **h2–10:** WebSocket server + `/health` + `/debug/message` (FR-B1, FR-B9); claim extraction + triage (FR-B3, FR-B4)
- ☐ **h10 Integration 1:** receive Member 1's real message, send back a verdict
- ☐ **h10–18:** detector orchestration + aggregation (FR-B5, §8.4); SQLite ledger (FR-B6); heat + levels (FR-B7, §9)
- ☐ **h18–24:** send `bubble.content` using Member 4's module (FR-B8); **Pushback Detector** (FR-C4) + tests; source-document tracking (FR-B11)
- ☐ **h24–30:** MongoDB learning store (FR-L1); ElevenLabs voice + cache (FR-V1, FR-V2); performance: caching, timeouts, latency logging (FR-B10)
- ☐ **h30–36:** stability fixes; be on call during demo rehearsals

Member 3 — Detectors (Python)

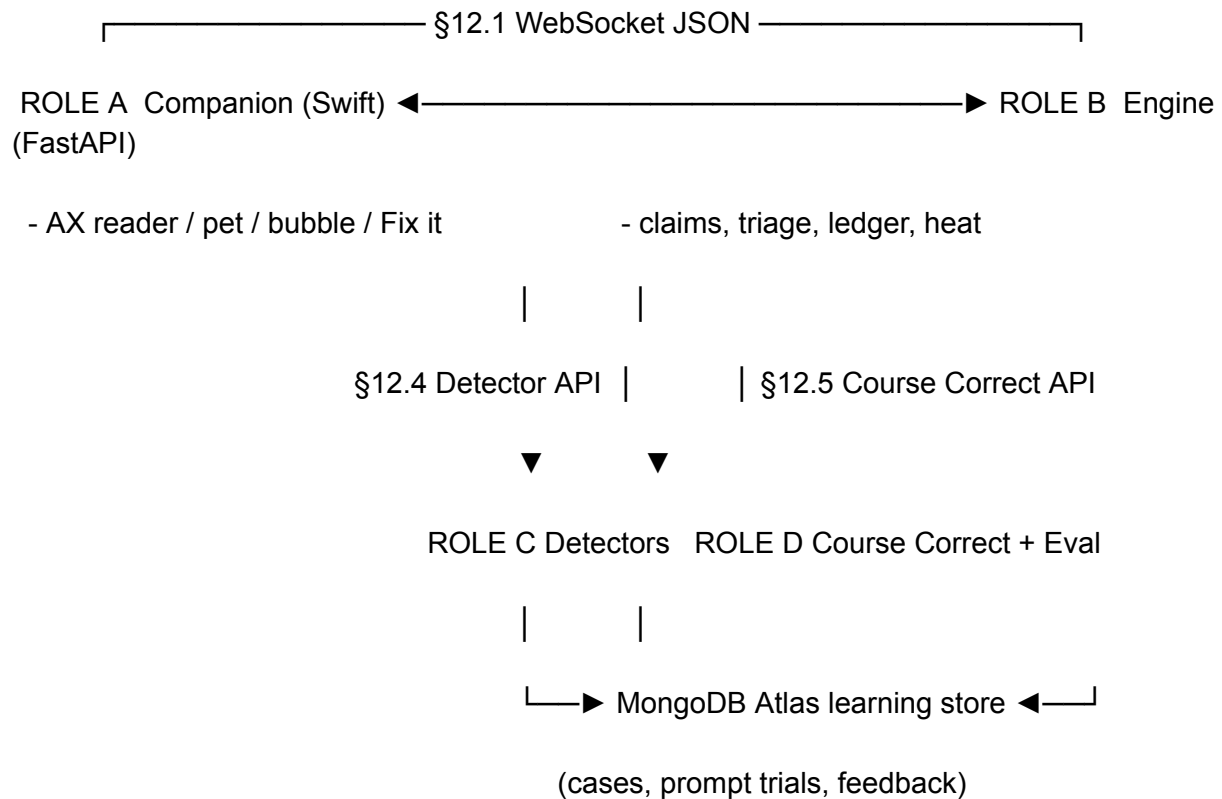
- ☐ **Before the event:** get Anthropic + Tavily keys; run `check_apis.py`; try Crossref + Semantic Scholar by hand; try `inspect.signature` on a few library functions
- ☐ **h0–2:** agree on the detector interface (§12.4) with Member 2; write `detectors/base.py`
- ☐ **h2–10:** Reference Auditor: papers, URLs, packages (FR-C1) + tests
- ☐ **h10–18:** Claim Verifier: search + judge (FR-C2) + tests
- ☐ **h18–24:** Consistency Probe (FR-C3) + tests
- ☐ **h24–30:** Code API Checker (FR-C7) + tests; tune thresholds with Member 4's eval results
- ☐ **h28–30 (only if Code API Checker is done):** experience memory with Atlas Vector Search (FR-L3)
- ☐ **h30–36:** bug fixes only; help rehearse

Member 4 — Course Correct + Eval + Pitch

- ☐ **Before the event:** draft the pitch outline; collect niche topics that make AIs invent papers
- ☐ **h0–2:** run the team kickoff: confirm names in §6, answer the open questions in §19
- ☐ **h2–10:** write ~38 trap prompts with labels (§17); **Source Faithfulness Checker** (FR-C6) + tests

- ☐ **h10–18:** diagnosis (FR-D1); prompt builder + templates for all 4 prompt types and all 6 hallucination types, **3 variants each** for the bandit (FR-D2, FR-D3, FR-L2)
- ☐ **h18–24:** bubble copy per level (FR-D4); hand module to Member 2
- ☐ **h24–30:** fix-rate tracking (FR-D5) + bandit updates (FR-L2); dataset seeding + seeded learning rounds + learning chart (FR-L6, FR-L7); run eval, write `eval/results.md` (FR-D6); record backup demo video; **call the feature freeze at h30**
- ☐ **h30–36:** finish deck with eval numbers + other-animal mockups (FR-D7); lead 3 full demo rehearsals

Role interaction map



7. Functional Requirements

Priority: **P0** = must ship · **P1** = should ship · **P2** = stretch

7.0 The 6 main hallucination types Witness detects and corrects ★

#	Type	Example	Detector (owner)	Correction prompt focus
1	Fake references	A paper, DOI, link or package that doesn't exist	Reference Auditor, FR-C1 (Member 3)	Only cite sources you're confident exist, or say you're unsure
2	False checkable facts	Wrong date, number, name or statistic that a search can disprove	Claim Verifier, FR-C2 (Member 3)	Quote the claim + the evidence; re-check and fix anything that depended on it
3	Made-up niche facts	Confident answers no search can settle; re-asked answers disagree	Consistency Probe, FR-C3 (Member 3)	Permission to say "I don't know"; mark uncertain claims
4	Caving under pushback	Flips a correct answer after "are you sure?" with no new evidence	Pushback Detector, FR-C4 (Member 2)	Judge on evidence only; standing firm is fine if it was right
5	Unfaithful summaries of long documents	Summary of a pasted article adds a number, name or claim that isn't in it, or changes one	Source Faithfulness Checker, FR-C6 (Member 4)	Answer only from the document; quote the passage for each claim; say "not in the document"
6	Made-up code APIs	A function, method or parameter that doesn't exist in the library, or	Code API Checker, FR-C7 (Member 3)	Name the library version; use only documented functions; here's

#	Type	Example	Detector (owner)	Correction prompt focus
		belongs to a different version		the real signature

Demo priority: 1 (fake reference) is the opening moment; 5, 6 and 4 follow (§11).

7.1 Companion (Role A)

ID	Requirement	Priority
FR-A1	Listen for <code>NSWorkspace.didActivateApplicationNotification</code> . Show pet when Claude app (bundle ID <code>com.anthropic.claudefordesktop</code>) is frontmost; hide (fade) within 300 ms otherwise.	P0
FR-A2	Pet window: borderless, transparent, non-activating <code>NSPanel</code> at <code>.floating</code> level; never steals focus. Positioned at bottom-right of Claude's window (inset 24 px); repositions on window move/resize (AXObserver or 250 ms poll). User can drag to another corner; remember corner.	P0
FR-A3	On attach, set <code>AXManualAccessibility = true</code> on Claude's app element. Walk the AX tree; extract ordered message texts, joining text fragments (split at bold/italic) into whole messages. Distinguish user vs assistant messages (by	P0

ID	Requirement	Priority
	tree structure discovered in Day-One Test; document the rule in /companion/README.md).	
FR-A4	Detect when an assistant reply is complete : text unchanged for ≥ 1.5 s after last change. Only then send message.new . Assign stable message_id = SHA-1 of (conversation position + first 200 chars). Never send the same message_id twice.	P0
FR-A5	WebSocket client to ws://<ENGINE_HOST>:8765/ws (default 127.0.0.1 , configurable). Auto-reconnect with backoff $1\text{ s} \rightarrow 2\text{ s} \rightarrow 5\text{ s}$ (max). Send session.start on connect.	P0
FR-A6	Pet renders 5 levels + Recovered exactly per §10 table using SwiftUI shapes (eyes, eyebrows, mouth as separate views; animate with withAnimation(.spring)). Level driven only by heat.update.level .	P0
FR-A7	Accessories: sweat drop (level ≥ 3), red flag (level ≥ 3), "?" badge with unverified count (level 1–2), "START FRESH?" sign (level 4). Number badge with heat always visible (color is never the only signal).	P0

ID	Requirement	Priority
FR-A8	Click pet → speech bubble (max width 320 px) showing <code>bubble.content</code> . Buttons: Fix it (if <code>correction</code> present), Details (expands panel with per-claim list + evidence links), I disagree (sends <code>feedback.disagree</code> for the top claim), Dismiss .	P0
FR-A9	Fix it: if message box is non-empty, ask "Replace or add to your text?". Activate Claude, focus the input (AX focus or click its frame), save clipboard, set clipboard to prompt, post Cmd+V, restore clipboard after 300 ms. Never post Enter . Send <code>correction.inserted</code> .	P0
FR-A10	First launch: onboarding sheet explaining why Accessibility permission is needed ("Witness only reads Claude, only types when you click Fix it, never sends messages"), with a button that opens the Accessibility pane.	P1
FR-A11	Mock mode (<code>WITNESS MOCK=1</code>): companion cycles through <code>/shared/fixtures/engine_*.json</code> so pet/bubble can be built without the engine.	P0
FR-A12	Direct AX insert (set <code>AXValue</code>) before paste;	P2

ID	Requirement	Priority
	verify it registered, else fall back to paste.	
FR-A13	ChatGPT desktop app support.	P2

7.2 Engine (Role B)

ID	Requirement	Priority
FR-B1	FastAPI app: <code>WS /ws</code> , <code>GET /health</code> → <code>{"ok": true, "version": "1.0"}</code> . Single session per connection.	P0
FR-B2	JSON Schemas in <code>/shared/schemas/</code> for every message in §12; Pydantic v2 models in <code>/engine/app/models.py</code> mirroring them exactly. Validate every inbound message; reply <code>error</code> message on invalid input.	P0
FR-B3	Claim extraction: LLM call converts an assistant reply into atomic, checkable claims (§12.3). Max 12 claims per reply. Each claim keeps <code>quote</code> = exact substring from the reply. Also extract <code>references</code> (papers, URLs, packages) as separate claims with <code>type</code> .	P0
FR-B4	Triage: assign <code>risk</code> (<code>high/medium/low</code>) — specific numbers, dates, names, citations, package	P0

ID	Requirement	Priority
	names = high; general explanations = low. Route: <code>reference</code> claims → Reference Auditor; <code>high/medium</code> factual → Claim Verifier; <code>high</code> factual with no evidence found → Consistency Probe; <code>source_summary</code> claims (reply is about a pasted document) → Source Faithfulness Checker; <code>code_api</code> claims (Python code blocks) → Code API Checker. Every user message that matches pushback patterns triggers the Pushback Detector on the next assistant reply. <code>low</code> claims are not checked.	
FR-B5	Run detectors concurrently (<code>asyncio.gather</code>) with per-detector timeout 12 s. Aggregate per §8.4.	P0
FR-B6	SQLite ledger (<code>witness.db</code>): tables <code>messages</code> , <code>claims</code> , <code>verdicts</code> , <code>corrections</code> , <code>feedback</code> . Every verdict stores evidence and detector name.	P0
FR-B7	Heat score and level per §9. Recompute after each verdict batch, each clean reply, each fix, each disagree.	P0
FR-B8	After processing a reply, send <code>verdicts.update</code> , then	P0

ID	Requirement	Priority
	<code>heat.update</code> , then <code>bubble.content</code> (which includes the Course Correct prompt when level ≥ 1).	
FR-B9	<code>POST /debug/message</code> accepts the same body as <code>message.new</code> and returns the full engine output as JSON (for teammates without a Mac).	P0
FR-B10	Structured logging (JSON lines) to <code>engine/logs/engine.log</code> with <code>session_id</code> , <code>message_id</code> , timings per stage.	P1
FR-B11	Source documents: any user message longer than 1,500 characters is stored as a <code>SourceDoc</code> in the session. When the next assistant reply is about it (judge LLM, or the user message contains summarize/explain/according to), claims from that reply get <code>type: "source_summary"</code> and <code>source_ref</code> = the doc id.	P0

7.3 Detectors (Role C, plus FR-C4 by Member 2 and FR-C6 by Member 4)

ID	Requirement	Priority
FR-C1	Reference Auditor. For <code>paper</code> claims: query Crossref (https://api.crossref.org/works?query.bibliographic=...&rows=5) and	P0

ID	Requirement	Priority
	Semantic Scholar; match title similarity ≥ 0.85 (normalized, case-insensitive) AND at least one author surname match AND year $\pm 1 \rightarrow$ supported ; no match $\rightarrow$ contradicted ("no such paper found"); partial match $\rightarrow$ unverified . For url claims: HTTP GET (5 s timeout, follow redirects); 404/DNS failure $\rightarrow$ contradicted ; if the reply attributes a quote/fact to the URL, check page text contains it (fuzzy) $\rightarrow$ else unverified . For package claims: PyPI JSON API / npm registry; missing $\rightarrow$ contradicted .	
FR-C2	Claim Verifier. Search (Tavily or Brave; 5 results) + Wikipedia REST summary. Judge LLM (temperature 0) compares claim to evidence snippets and returns supported / contradicted / unverified + a verbatim evidence quote + source URL. If no evidence found $\rightarrow$ unverified (never contradicted without evidence).	P0
FR-C3	Consistency Probe. For triaged claims only: turn claim into a question, sample 5 answers (temperature 1.0) in parallel, group by meaning	P0

ID	Requirement	Priority
	via judge LLM (temperature 0, JSON output), compute normalized semantic entropy (§8.3). $> 0.6 \rightarrow$ likely_hallucination , $0.3-0.6 \rightarrow$ uncertain , $< 0.3 \rightarrow$ consistent . Also report whether the original answer is in the majority group.	
FR-C4	Pushback Detector (Owner: Member 2). Trigger when a user message matches pushback patterns ("are you sure", "I think you're wrong", "that's not right", etc.) and contains no new evidence (no URL, no quoted source, no numbers not previously in the chat). Compare the previous assistant answer vs. the new one with a judge LLM: reversed / refined / maintained . reversed with no new evidence $\rightarrow$ flag caved_without_evidence .	P0
FR-C5	All detectors: implement §12.4 interface, respect timeout, cache identical lookups for the session (in-memory dict), never raise — return status: "error" with message.	P0
FR-C6	Source Faithfulness Checker (Owner: Member 4). For each source_summary claim: split the source doc	P0

ID	Requirement	Priority
	into ~800-token chunks, pick the 3 most relevant chunks (keyword overlap is enough), and ask the judge LLM (temperature 0, JSON) whether the claim is supported (return the supporting quote), contradicted (return the conflicting quote) or not_in_source. Specific numbers, names and dates missing from the source → not_in_source with confidence ≥ 0.8. Never uses web search.	
FR-C7	Code API Checker (Owner: Member 3), Python only. Parse each Python code block with ast (never execute it). Collect imports and every module.attr... call and its keyword arguments. For each library: if not installed in the engine's environment → check PyPI existence only (FR-C1) and return unverified. If installed: importlib.import_module, walk the attribute chain with getattr; missing attribute → nonexistent_api; for callables, inspect.signature and check every keyword argument exists (unless the signature accepts	P0

ID	Requirement	Priority
	**kwargs) → unknown keyword → nonexistent_api. Evidence = installed library version + the real signature or a "did you mean" suggestion (difflib.get_close_matches on dir(obj)). Pre-install in the engine venv: requests, numpy, pandas, fastapi, pydantic, httpx, flask, anthropic, openai.	

7.4 Course Correct (Role D)

ID	Requirement	Priority
FR-D1	Diagnosis produces a DriftProfile (§12.5): failures list, blast radius (claims that depend on a failed claim — judge LLM over the ledger), trend (isolated / worsening), and root cause from the pattern table (§8.5).	P0
FR-D2	Prompt builder always produces the 5-part structure: (1) what's wrong, quoted + evidence; (2) the pattern, neutral wording; (3) exactly what to redo, nothing more; (4) guardrails for the rest of the chat; (5) answer format (mark claims [verified] / [unverified]). Length by level (§10).	P0

ID	Requirement	Priority
FR-D3	Four correction rules enforced in the builder's system prompt: quote the exact problem; show the evidence; ask it to re-check, not to agree (explicitly allow "I was right"); allow "I don't know". No accusatory tone.	P0
FR-D4	Bubble copy : headline (≤ 60 chars), up to 3 problems (≤ 120 chars each, most severe first), "how it's going off course" (≤ 200 chars), confidence label (Very sure / Fairly sure / Not sure) with reason, action line.	P0
FR-D5	Fix-rate tracking : on correction.inserted , mark the correction pending; when the next assistant reply arrives, re-run the relevant detectors on the originally failed claims; store fixed: true/false .	P1
FR-D6	Evaluation harness eval/run_eval.py : runs trap prompts through the engine via /debug/message with recorded model answers; outputs precision, recall, fix rate, median latency to eval/results.md .	P0
FR-D7	Pitch deck + backup demo video recorded by hour 30.	P0

7.5 Learning Loop (how Witness improves)

What "learning" means here — say it exactly this way in the pitch: Witness learns *which correction prompts work* (reinforcement learning with a multi-armed bandit), *remembers confirmed hallucinations* to judge new ones better (experience memory), and *tunes its own alarm thresholds* from user feedback. It does **not** train or fine-tune a neural network. Never call it an "RL-trained model".

flowchart LR

D[Detector verdict] --> C{Confirmed?}

C -- evidence-backed / user-confirmed / verified fix --> M[(MongoDB Atlas)]

C -- unconfirmed guess --> X[Not learned from]

M --> B[Bandit picks best prompt variant]

M --> E[Experience memory: similar past cases]

M --> T[Threshold calibration]

B --> P[Course Correct prompt]

P --> R[Next reply re-checked = reward]

R --> M

E --> D

T --> D

ID	Requirement	Owner	Priority
FR-L1	Learning store. MongoDB Atlas (free tier) with the collections in §12.7. Writes are async and never block verdicts; if Atlas is unreachable, queue	Member 2	P0

ID	Requirement	Owner	Priority
	records locally (JSON lines) and retry. Only anonymized records (§12.7) — never full conversations or pasted documents. Learning is opt-in in onboarding (default on for the demo machine).		
FR-L2	Prompt-learning bandit (reinforcement learning). For each (<i>failure_type</i> , <i>prompt_type</i>) there are 3 prompt variants (e.g. evidence-first, question-first, checklist-style). Each variant keeps a $\text{Beta}(\alpha, \beta)$ distribution starting at $\text{Beta}(1, 1)$. Thompson sampling: sample from each variant's Beta, use the highest. Reward: FR-D5 re-check says fixed $\rightarrow \alpha += 1$; not fixed $\rightarrow \beta += 1$; no next reply within 10 min $\rightarrow$ no update. Stored in <i>prompt_variants</i> .	Member 4	P0
FR-L3	Experience memory. Embed the <i>normalized</i> text of every confirmed case	Member 3	P1

ID	Requirement	Owner	Priority
	(§12.7 cases). Before a judge LLM call (Claim Verifier, Source Faithfulness, Pushback), retrieve the 3 most similar confirmed cases with Atlas Vector Search (same claim type, cosine similarity ≥ 0.75) and include them as labeled examples in the judge prompt. If Atlas or embeddings fail, skip silently.		
FR-L4	Threshold calibration. Per detector, track the share of red flags users marked "I disagree" over the last 20 flags. If $> 30\%$, raise that detector's red threshold (e.g. not_in_source confidence, entropy cutoff) by 0.05, max +0.15; if $< 10\%$ for 20 flags, lower it by 0.05 back toward the default. Stored in detector_calibration .	Member 4	P1
FR-L5	Confirmed-only rule. Only learn from: (a) verdicts backed by external evidence	All	P0

ID	Requirement	Owner	Priority
	(Crossref/PyPI lookup, search quote, source-document quote, library signature); (b) user feedback (I disagree, or a fix the user accepted); (c) verified fixes (FR-D5); (d) labeled dataset examples (FR-L6). Never learn from likely_hallucination/uncertain alone. This prevents Witness from reinforcing its own mistakes.		
FR-L6	Dataset seeding (replaces web scraping). eval/seed_datasets.py imports ~200 labeled examples from public datasets — HaluEval (QA + summarization), TruthfulQA, FEVER — into cases with origin: "dataset". Download the datasets; do not scrape websites.	Member 4	P0
FR-L7	Learning chart. eval/learning_rounds.py runs the trap set (§17) for ≥ 5 rounds against	Member 4	P0

ID	Requirement	Owner	Priority
	recorded model responses, updating the bandit after each round, and plots Course Correct fix rate per round (matplotlib PNG for the deck). Label the chart "seeded evaluation rounds".		

Honest limits (say these if judges ask): in 36 hours Witness sees too few real hallucinations to learn much from live use, so the learning curve comes from seeded rounds. The bandit needs roughly 10–20 trials per variant before its choice is reliable.

7.6 Voice (ElevenLabs)

ID	Requirement	Owner	Priority
FR-V1	Engine generates speech with the ElevenLabs text-to-speech API (key in <code>ELEVENLABS_API_KEY</code> , voice ID in <code>WITNESS_VOICE_ID</code>) and sends a <code>voice.play</code> message (§12.2).	Member 2	P1
FR-V2	When the pet speaks: only when the level rises to 3 (Alarmed) or 4 (Meltdown), when the user opens the bubble, and on Recovered ("Fixed it!"). At most one line	Member 2	P1

ID	Requirement	Owner	Priority
	every 20 s. Lines $\leq$ 15 words, taken from the bubble headline. Pre-generate and cache ~20 common lines at startup (e.g. "Hmm, that paper doesn't exist.") so playback is instant.		
FR-V3	Companion plays the audio (AVAudioPlayer), shows a speaker icon while talking, and has a mute toggle (remembered). If audio fails, fall back to macOS AVSpeechSynthesizer, or stay silent.	Member 1	P1

8. Detection Logic Details

8.1 Verdict statuses (exact enum — use these strings everywhere)

supported · contradicted · unverified · likely_hallucination · uncertain · consistent · caved_without_evidence · not_in_source · nonexistent_api · error

8.2 Claim types (exact enum)

fact · paper · url · package · number · code · code_api · source_summary · other

8.3 Semantic entropy formula

Given N answers grouped into meaning groups with sizes n_i :

$$p_i = n_i / N$$

$$H = -\sum p_i \cdot \ln(p_i)$$

$$H_{\text{norm}} = H / \ln(N) \quad \# 0 = \text{all agree}, 1 = \text{all different}$$

8.4 Aggregation into a final claim status (red / amber / green)

Final	Rule
 red	Any contradicted with evidence , OR likely_hallucination AND Claim Verifier found no supporting evidence, OR caved_without_evidence , OR nonexistent_api , OR not_in_source with confidence ≥ 0.8
 amber	unverified, uncertain, likely_hallucination alone, or not_in_source with confidence < 0.8
 green	supported or consistent with no red/amber signal
 skipped	Low-risk claims or detector error

Precision rule: a claim is red only if an independent signal (evidence or strong disagreement + no evidence) backs it. When in doubt → amber, not red.

8.5 Drift patterns (root cause → what the prompt adds)

Pattern ID	Signs	Prompt adds
knowledge_gap	Fabrications on niche specifics; high entropy	Permission to say "I don't know"; mark unverified claims
anchored_wrong_assumption	Later claims depend on one false claim	Name the false premise; redo only dependent steps
caving	Answer flips after pushback, no new evidence	Judge on evidence only; standing firm is fine
fabricated_sources	≥ 1 fake paper/URL/package	Only cite sources you're confident exist, else say so

Pattern ID	Signs	Prompt adds
<code>outdated_knowledge</code>	Wrong facts about recent events/versions/prices	Flag possible staleness; ask for dates + uncertainty
<code>source_drift</code>	Summary claims not in / contradicting the pasted document	Answer only from the document; quote the passage for each claim; say "not in the document"
<code>api_confusion</code>	Made-up or wrong-version functions/parameters	Name the library version assumed; use only documented functions; include the real signature

9. Heat Score (exact algorithm)

Heat is an integer 0–100 per session, clamped.

Adds

Event	Heat
Claim final status <code>red</code> (false fact, fake reference, <code>nonexistent_api</code> , source contradiction)	+25
<code>caved_without_evidence</code>	+15
Claim final status <code>amber</code>	+8

Subtracts

Event	Heat
Assistant reply fully processed with zero red/amber claims	−10
Correction verified as fixed (FR-D5)	−20

Event	Heat
User clicks I disagree on a claim	remove that claim's contribution + mark in ledger
Time decay	−1 per 30 s since last heat increase

Levels (with hysteresis: level changes only after the new level's range holds for 2 s)

Level	Name	Heat range
0	Calm	0–10
1	Curious	11–30
2	Concerned	31–55
3	Alarmed	56–80
4	Meltdown	81–100
R	Recovered	Shown for 3 s after a verified fix, then the computed level

10. Pet Behavior Spec (Role A renders, Role D writes copy)

Level	Eyes	Eyebrows	Mouth	Action / Accessory	Bubble shows prompt?	Prompt type
0 Calm	Soft, slow blink every 4 s	Relaxed	Small smile	Gentle breathing	No	(Pre-send tip — stretch)
1 Curious	Normal, one eye slightly larger	One raised	Neutral	Head tilt, "?" badge with count	On click	Verify nudge (1–2 sentences)
2 Concerned	Narrowed / side-eye	Knitted	Flat line	Fidget, "?" badge	Offered in bubble	Targeted correction

Level	Eyes	Eyebrows	Mouth	Action / Accessory	Bubble shows prompt?	Prompt type
						(short paragraph)
3 Alarmed	Wide	High	Open "O"	Sweat drop + waving red flag	Offered immediately	Diagnostic reset (1–2 paragraphs)
4 Meltdown	Spirals	Extreme	Wavy	Steam puffs, "START FRESH?" sign	Offered immediately	Fresh-start hand-off (structured block for a new chat)
R Recovered	Sparkle / happy arcs	Raised	Big grin	Hop	No	—

Bubble tone by level: Curious = light ("Hmm, I couldn't confirm two claims."), Alarmed = direct and specific, Meltdown = recommends starting fresh and says why.

Example bubble (level 3):

Heads up: Claude is making up sources. • The paper "Lee & Park, 2022" doesn't exist in Crossref. • It said 500 requests/min; the docs say 100. *How it's going off course:* it's filling gaps with confident guesses, and steps 4–6 build on the wrong number. *Very sure* — checked Crossref and the official docs. [Fix it] [Details] [I disagree] [Dismiss]

Example level-3 prompt (Diagnostic reset):

I want to pause and fix a few problems before we continue.

1. You cited "Lee & Park, 2022, *IEEE Access*"; it doesn't exist in Crossref. 2. You said the API limit is 500 requests/min; the official docs say 100 [link]. These look connected: steps 4–6 of your plan rely on the 500/min figure. Please redo only those steps using 100/min. Going forward: only cite sources you're confident exist, and mark each factual claim [verified] or [unverified]. If you're unsure about something, it's fine to say so.

11. Demo Script (3 minutes — must run 3× in a row before hour 32)

1. Open Claude app → pet appears on the window corner, **Calm**. Ask for research papers on a niche topic. Pet goes **Alarmed**, waves red flag.
2. Click pet → bubble explains: one paper doesn't exist (Crossref evidence); AI is filling gaps with confident guesses.
3. Click **Fix it** → level-matched prompt appears in Claude's message box → press Enter → Claude corrects itself → pet shows **Recovered**.
4. **Summary**: paste a short article and ask for a summary → pet flags "this number isn't in your article" (Source Faithfulness).
5. **Code**: ask for Python using a library → pet flags a function that doesn't exist, showing the real signature (Code API Checker).
6. **Caving**: push back with "Are you sure?" on a correct answer → AI flips → pet flags it and offers a neutral re-check prompt (Pushback Detector).
7. Close: eval numbers (precision, fix rate) → **learning chart** ("Course Correct's fix rate rose from X% to Y% over 5 seeded rounds as it learned which prompts work") → slide with the other 3 animals + roadmap.

Voice: in step 1 the pet says the headline out loud when it goes Alarmed, and says "Fixed it!" on Recovered (step 3). Keep the laptop volume up; mute is one click if the room is noisy.

If time is tight on stage, run steps 1–3 plus the **two** of steps 4–6 that triggered most reliably in rehearsal (use the Consistency Probe as a backup moment). Turn **web search off** in Claude for the demo.

Backup: pre-recorded video of the same flow (Role D, by hour 30).

12. Interface Contracts ⚠ (change only with whole-team agreement)

All JSON: UTF-8, **snake_case** keys, ISO-8601 UTC timestamps ("2026-10-01T14:03:22Z"). The `//` comments in the examples below are annotations only; real messages must be plain JSON with no comments. Every WebSocket message has this envelope:

```
{
```

```
"type": "message.new",  
  
"session_id": "uuid-v4",  
  
"ts": "2026-10-01T14:03:22Z",  
  
"payload": { }  
  
}
```

12.1 Companion → Engine

session.start

```
{ "app": "claude", "app_version": "unknown", "companion_version": "1.0" }
```

message.new

```
{  
  
  "message_id": "sha1-hex",  
  
  "role": "assistant",          // "assistant" | "user"  
  
  "text": "full message text",  
  
  "position": 7                 // index in conversation, 0-based  
  
}
```

correction.inserted

```
{ "correction_id": "uuid-v4" }
```

feedback.disagree

```
{ "claim_id": "uuid-v4", "note": "optional free text" }
```

12.2 Engine → Companion

verdicts.update

```
{
```

```
"message_id": "sha1-hex",

"claims": [

  {

    "claim_id": "uuid-v4",

    "quote": "exact substring from the reply",

    "type": "paper",

    "risk": "high",

    "final": "red",           // "red" | "amber" | "green" | "skipped"

    "detector_results": [

      {

        "detector": "reference_auditor", // "reference_auditor" | "claim_verifier" |
"consistency_probe" | "pushback" | "source_faithfulness" | "code_api_checker"

        "status": "contradicted",

        "confidence": 0.95,

        "evidence": [

          { "source": "Crossref", "url": "https://api.crossref.org/...", "snippet": "No matching work
found" }

        ],

        "explanation": "No paper with this title and authors exists in Crossref or Semantic
Scholar.",

        "latency_ms": 1840

      }

    ]

  }
```

```
}  
  
]  
  
}
```

heat.update

```
{ "heat": 62, "level": 3, "recovered": false, "red_count": 2, "amber_count": 1 }
```

bubble.content

```
{  
  
  "level": 3,  
  
  "headline": "Heads up: Claude is making up sources.",  
  
  "problems": [  
  
    { "claim_id": "uuid-v4", "text": "The paper 'Lee & Park, 2022' doesn't exist.", "evidence_url":  
      "https://..." },  
  
  ],  
  
  "pattern_text": "It's filling gaps with confident guesses instead of saying it doesn't know.",  
  
  "confidence_label": "Very sure",  
  
  "confidence_reason": "Checked Crossref and Semantic Scholar.",  
  
  "action_text": "I wrote a prompt to fix this.",  
  
  "correction": {  
  
    "correction_id": "uuid-v4",  
  
    "prompt_type": "diagnostic_reset",    // "verify_nudge" | "targeted_correction" |  
    "diagnostic_reset" | "fresh_start"  
  
    "text": "I want to pause and fix a few problems..."  
  }  
}
```

```
}  
  
}
```

`correction` is `null` at level 0.

voice.play (FR-V1)

```
{ "text": "Heads up, that paper doesn't exist.", "audio_b64": "<base64 mp3>", "mime":  
"audio/mpeg", "level": 3 }
```

error

```
{ "code": "invalid_message", "message": "human-readable detail" }
```

12.3 Internal **Claim** model (Role B defines, Role C/D consume)

class Claim(BaseModel):

```
    claim_id: str          # uuid4  
  
    message_id: str  
  
    quote: str            # exact substring of the reply  
  
    normalized: str       # standalone restatement, e.g. "The Eiffel Tower was completed in  
1889."  
  
    type:  
    Literal["fact", "paper", "url", "package", "number", "code", "code_api", "source_summary", "other"]  
  
    risk: Literal["high", "medium", "low"]  
  
    question: str | None  # question form for the consistency probe  
  
    context: str          # the user message that prompted this reply (truncated to 1,000 chars)  
  
    source_ref: str | None # SourceDoc id when type == "source_summary" (FR-B11)  
  
    code: str | None      # the code block when type == "code_api"
```

class SourceDoc(BaseModel):

```

doc_id: str          # uuid4

message_id: str      # the user message it came from

text: str            # full pasted text (kept locally only)

```

12.4 Detector interface (Role C implements, Role B calls)

engine/app/detectors/base.py

```
class DetectorResult(BaseModel):
```

```

    detector: Literal["reference_auditor", "claim_verifier", "consistency_probe", "pushback",
                      "source_faithfulness", "code_api_checker"]

    status: Literal["supported", "contradicted", "unverified", "likely_hallucination",
                   "uncertain", "consistent", "caved_without_evidence", "not_in_source",
                   "nonexistent_api", "error"]

    confidence: float    # 0.0–1.0

    evidence: list[Evidence] # Evidence(source: str, url: str | None, snippet: str)

    explanation: str     # one plain-English sentence

    latency_ms: int

```

```
class Detector(Protocol):
```

```

    name: str

    async def check(self, claim: Claim, session: SessionContext) -> DetectorResult: ...

```

Rules: must finish within the timeout passed by the engine, must never raise, must be deterministic given cached lookups.

12.5 Course Correct interface (Role D implements, Role B calls)

```
class DriftProfile(BaseModel):
```

```

    failures: list[str]    # claim_ids that are red/amber

```


blast_radius: list[str] # claim_ids that depend on failures

trend: Literal["isolated","worsening"]

root_causes: list[Literal["knowledge_gap","anchored_wrong_assumption","caving",
 "fabricated_sources","outdated_knowledge",
 "source_drift","api_confusion"]]

def diagnose(session: SessionContext) -> DriftProfile: ...

def build_bubble(level: int, profile: DriftProfile, session: SessionContext) -> BubbleContent: ...

12.6 Fixtures

`/shared/fixtures/` holds example JSON for every message type, plus 7 full scenarios (`scenario_fake_citation.json`, `scenario_false_fact.json`, `scenario_niche_entropy.json`, `scenario_pushback.json`, `scenario_summary_drift.json`, `scenario_code_api.json`, `scenario_clean.json`).

Role B publishes these by hour 4. Role A's mock mode and Role C/D tests use them.

12.7 MongoDB Atlas collections (FR-L1) — anonymized records only

Database: `witness`. No conversation text, no pasted documents, no user names.

cases — confirmed hallucinations and confirmed-correct claims

```
{  
  
  "_id": "uuid-v4",  
  
  "normalized": "The paper 'Lee & Park, 2022, IEEE Access' exists.",  
  
  "claim_type": "paper",  
  
  "detector": "reference_auditor",  
  
  "final": "red",  
  
  "confirmed_by": "evidence",            // "evidence" | "user_feedback" | "verified_fix" | "dataset"  
  
  "origin": "live",                    // "live" | "dataset" | "eval"
```

```
"evidence_snippet": "No matching work found in Crossref",  
  
"embedding": [0.012, -0.03],           // vector for Atlas Vector Search (FR-L3)  
  
"app": "claude",  
  
"created_at": "2026-10-01T14:03:22Z"  
  
}
```

Vector index **cases_vector** on **embedding** (cosine), with filter field **claim_type**.

prompt_variants — bandit state (FR-L2)

```
{ "_id": "fabricated_sources:diagnostic_reset:v2", "failure_type": "fabricated_sources",  
  
  "prompt_type": "diagnostic_reset", "variant": "v2", "template": "...", "alpha": 7, "beta": 3,  
  
  "updated_at": "2026-10-01T14:03:22Z" }
```

prompt_trials — one row per inserted correction

```
{ "_id": "correction_id", "variant_id": "fabricated_sources:diagnostic_reset:v2", "level": 3,  
  
  "fixed": true, "origin": "live", "created_at": "2026-10-01T14:03:22Z" }
```

feedback — "I disagree" clicks

```
{ "_id": "uuid-v4", "detector": "source_faithfulness", "claim_type": "source_summary",  
  
  "confidence": 0.82, "created_at": "2026-10-01T14:03:22Z" }
```

detector_calibration — current thresholds (FR-L4)

```
{ "_id": "source_faithfulness", "red_threshold": 0.85, "recent_disagree_rate": 0.35,  
  
  "updated_at": "2026-10-01T14:03:22Z" }
```

13. Risks and Fallbacks

ID	Risk	Owner	Fallback / Decision point
R1	Claude app text not readable via Accessibility API	A	Decide by hour 4 : try ScreenCaptureKit + Vision OCR; if still unreliable → Chrome extension version of the same pet on claude.ai (engine unchanged). Day-One Test already passed on Mayank's Mac.
R2	Paste into Claude's message box fails	A	Fix it copies prompt to clipboard + bubble says "Press Cmd+V in Claude"
R3	Too many false alarms	C/D	Aggregation rule §8.4 (red needs independent evidence); I disagree lowers heat
R4	Verdicts too slow	B	Triage, concurrency, timeouts, caching; cap at 12 claims/reply
R5	Correction prompt makes the AI cave	D	Neutral wording, explicit "standing firm is fine"; measure fix rate vs. a plain "that's wrong, fix it"
R6	API rate limits / cost	B/C	Cache lookups; only probe high-risk claims; keep 5 samples max

ID	Risk	Owner	Fallback / Decision point
R7	Live demo fails, or Claude refuses to hallucinate on cue	D	Backup video; scripted prompts tested in rehearsal to trigger each detector; web search off
R8	Claude app update changes AX structure	A	Keep element-matching rules in one config file <code>companion/Config/AXRules.json</code>
R9	Six detectors is a lot for 36 hours	All	Build order in §16. If behind at hour 24, cut in this order: experience memory (FR-L3), threshold calibration (FR-L4), voice (FR-V*), Code API Checker keyword-argument checks (keep attribute checks), then Consistency Probe. Never cut the bandit + learning chart (FR-L2, FR-L7).
R10	Witness learns from its own mistakes (self-reinforcing errors)	All	Confirmed-only rule (FR-L5); dataset and eval labels are ground truth
R11	Not enough live data for learning to show	D	Seeded rounds (FR-L6, FR-L7), clearly labeled; explain the honest limits (§7.5)

ID	Risk	Owner	Fallback / Decision point
R12	Voice is annoying or slow	B/A	Speak only at level ≥ 3 / bubble open / recovered, 1 line per 20 s, cached lines, mute button
R13	Privacy concern about cloud storage	B	Anonymized records only (§12.7), opt-in, conversations and documents stay local
R14	Atlas or ElevenLabs down at the venue	B	Local queue for Mongo writes; voice falls back to macOS speech or silence; demo never depends on either

14. Rules for AI Coding Assistants 🤖 (paste into every AI session)

1. **Read this entire PRD before writing code.** Follow §12 contracts exactly — field names, enums, types. Do not invent new fields; propose them to the team instead.
2. **Stay in your role's folders** (§6). Don't modify another role's files; if you need a change there, write it as a note in `/docs/requests.md`.
3. **Stack is fixed:** Swift 5.9+ / SwiftUI / AppKit / AVFoundation (macOS 14+) for `/companion`; Python 3.11, FastAPI, Pydantic v2, httpx, aiosqlite, motor (async MongoDB driver), elevenlabs, numpy, matplotlib, pytest for `/engine`. Don't add frameworks without team approval.
4. **Secrets:** read keys from environment variables only (`ANTHROPIC_API_KEY`, `OPENAI_API_KEY` (optional), `TAVILY_API_KEY` or `BRAVE_API_KEY`, `MONGODB_URI`, `ELEVENLABS_API_KEY`, `WITNESS_VOICE_ID`, `VOYAGE_API_KEY` or the agreed embedding key). Never hardcode or commit keys. `.env` is in `.gitignore`.
5. **LLM calls:** model name comes from env var `WITNESS_MODEL` (default: the team's agreed Claude model). Judge/grouping calls use temperature 0 and must request JSON-only output and parse defensively (strip code fences, retry once on parse failure).
6. **Never press Enter / never auto-send** in any companion code.

7. **Errors:** no uncaught exceptions across module boundaries. Detectors return `status: "error"`. Engine sends an `error` message. Companion shows a neutral state, never crashes.
 8. **Logging:** Python — `logging` with JSON formatter; Swift — `os.Logger(subsystem: "app.witness", category: ...)`. No `print` in committed code except scripts.
 9. **Style:** Python — type hints everywhere, `ruff + black` (line length 100). Swift — SwiftUI views small and composable; state in an `@Observable` view model.
 10. **Tests:** every detector and the heat calculator get pytest unit tests using `/shared/fixtures`. Write the test first when changing logic in §8 or §9.
 11. **Complete files:** when editing, output the complete updated file, not fragments.
 12. **Comments:** explain *why*, and reference requirement IDs (e.g. `# FR-C3: normalized semantic entropy`).
 13. **Privacy:** never send whole conversations to search APIs — only the claim's `normalized` text. Pasted source documents stay local (only sent to the judge LLM). Keep the ledger local.
 14. **Code API Checker safety:** never execute code from the AI's reply; only parse it with `ast` and import well-known installed libraries.
 15. **Learning:** only write to MongoDB what §12.7 allows. Only learn from confirmed signals (FR-L5). Never block a verdict on a MongoDB or ElevenLabs call.
 16. **Honest claims:** in code comments, README and slides, describe learning as bandit + memory + calibration — never as a trained model.
-

15. Repository, Workflow and Change Process

15.1 Repo layout

witness/

```
|— PRD.md           ← this file
|
|— README.md        ← setup + run instructions
|
|— .env.example
|
|— companion/       ← ROLE A (Xcode project)
|
| |— Witness.xcodeproj
|
| |— Witness/ (App, Pet, Bubble, AX, Networking, Config/AXRules.json)
```

```

|   └─ README.md           ← AX tree findings from Day-One Test

└─ engine/                 ← ROLE B (+ C in detectors/, D in course_correct/)

|   └─ app/

|       └─ main.py         ← FastAPI, /ws, /health, /debug/message

|       └─ models.py       ← Pydantic mirrors of §12

|       └─ extraction.py   triage.py aggregate.py heat.py ledger.py

|       └─ detectors/      ← ROLE C (pushback.py = B, source_faithfulness.py = D)

|       └─ course_correct/ ← ROLE D (incl. bandit.py)

|       └─ learning/       ← ROLE B store.py (Mongo), ROLE C memory.py (vector search),
ROLE D calibration.py

|       └─ voice.py        ← ROLE B (ElevenLabs)

└─ tests/

└─ pyproject.toml

└─ shared/

└─ schemas/               ← JSON Schemas (ROLE B steward)

└─ fixtures/              ← example messages + 7 scenarios

└─ eval/                  ← ROLE D (trap_prompts.jsonl, run_eval.py, seed_datasets.py,
learning_rounds.py, results.md)

└─ pitch/                 ← ROLE D (deck, animal mockups, demo video)

└─ docs/requests.md       ← cross-role change requests

```

15.2 Git workflow

- `main` is always demo-able. Work on branches `role-a/<feature>`, `role-b/<feature>`, etc.
- Small PRs; the owning role merges. Commit format: `FR-C1: add Crossref lookup`.

- Integration merges at hours **10, 18, 24, 30** (whole team, 15 min).
- ⚠ Before any `git pull --rebase`, `reset`, or force-push, commit or stash local work — these can overwrite local changes.

15.3 Change process for §12 contracts

1. Propose in team chat with the exact JSON diff.
 2. Role B updates `/shared/schemas + models.py` + fixtures in one commit.
 3. Everyone pulls; Role A updates Swift `Codable` structs.
-

16. Timeline and Milestones

Hours	Milestone	Owners	Checkpoint
0–2	Repo + <code>.env</code> set up; §12 schemas v1 published; Day-One Test run and reported	All; A runs test; B publishes schemas	✅ Go/no-go on AX reading
2–10	Pet window attaches to Claude + reads replies (A); engine WS + extraction + fixtures (B); Reference Auditor (C); trap prompt set + Source Faithfulness Checker (D)	All	Integration 1 @ h10: real reply → engine → any verdict back to pet
10–18	Claim Verifier (C); aggregation + heat (B); 5 expression levels (A); diagnosis + level-matched prompts (D)	All	Integration 2 @ h18: fake citation → pet Alarmed
18–24	Consistency Probe (C); Pushback Detector + source-doc tracking (B); bubble content	All	Integration 3 @ h24: click pet → correct bubble

Hours	Milestone	Owners	Checkpoint
	end to end (B/D); bubble UI (A)		
24–30	Fix it paste + voice playback (A); MongoDB store + ElevenLabs voice (B); Code API Checker, then experience memory if time (C); fix-rate tracking + bandit + dataset seeding + learning chart, run eval, record backup video (D)	All	Integration 4 @ h30: full demo flow works. FEATURE FREEZE
30–36	Bug fixes, polish, rehearse demo 3+ times, finalize deck	All	Demo runs 3× in a row

17. Evaluation Plan (Role D)

- **Trap set:** ~38 prompts in `eval/trap_prompts.jsonl`, each with a recorded model answer and ground-truth labels per claim:
 - 8 fake-citation bait ("list 5 papers on with DOIs")
 - 6 checkable facts (some answers correct, some wrong)
 - 5 niche/unanswerable facts (Consistency Probe)
 - 5 pushback cases (correct answer + "are you sure?" with no evidence)
 - 5 long-document summaries (pasted article + summary with 0–2 planted unsupported claims)
 - 5 Python code answers (some with made-up functions/parameters)
 - 4 clean controls (should stay green)
- **Metrics:** precision of red flags (target ≥ 0.80), recall (report as-is), fix rate of Course Correct prompts (target ≥ 0.60), median verdict latency (target ≤ 5 s). Report per hallucination type.
- **Baselines:** (a) ask the same model "are you sure?"; (b) plain "that's wrong, fix it" as the correction. Report Witness vs. both.

17.1 Learning evaluation (FR-L7)

- Run the trap set for ≥ 5 rounds; after each round, update the bandit with the verified outcomes.
 - Report: fix rate per round (chart), which variant won per failure type, and bandit fix rate vs. always using variant v1.
 - Hold out ~ 10 trap prompts that are never used for learning, and report the final fix rate on those too, so the improvement isn't just memorization.
-

18. Non-Functional Requirements

Area	Requirement
Latency	Verdict ≤ 8 s after reply completes (target ≤ 5 s); pet expression change animates in ≤ 400 ms
Privacy	Ledger stored locally only; search APIs receive only claim text; pasted documents never leave the machine except to the judge LLM; pause switch in the menu bar
Control	Never sends messages; never types without a click
Reliability	Companion survives engine restarts (auto-reconnect); engine survives detector failures
Accessibility	Heat number badge + face + action always change together (color never the only signal)
Cost	$\leq \sim 15$ LLM calls per assistant reply on average (triage keeps most claims low-risk)

19. Open Questions (decide by hour 2)

#	Question	Default if undecided
Q1	Which animal ships in the app?	Owl
Q2	Search API: Tavily or Brave?	Tavily
Q3	Which Claude model for extraction/judging/sampling?	One agreed model via WITNESS_MODEL
Q4	Engine runs on the Mac or on a teammate's laptop over Wi-Fi?	On the Mac (127.0.0.1) for the demo
Q5	Claude app bundle ID + message-role rule in the AX tree	Bundle ID confirmed: com.anthropic.claudefordesktop (Day-One Test). Message-role rule: Role A fills in. Note: text arrives in small fragments (split at bold/italic), so the companion must join fragments into whole messages.
Q6	Embedding model for experience memory (FR-L3)	Voyage AI (works well with Atlas Vector Search); fallback: skip FR-L3
Q7	ElevenLabs voice for the pet	Pick one friendly preset voice; store its ID in WITNESS_VOICE_ID

20. Glossary

- **Hallucination:** the AI confidently states something made up (a fact, number, source, package, code API) or not supported by the document it was given.
- **Caving:** the AI abandons a correct answer because the user pushed back, not because of new evidence.

- **Source faithfulness:** whether a summary only says things that are actually in the source document.
- **Heat score:** 0–100 measure of how badly the current session is going; drives the pet's level.
- **Course Correct:** the feature that diagnoses how the AI is drifting and writes a correction prompt.
- **Drift profile:** diagnosis output — what failed, what depends on it, trend, root cause.
- **Semantic entropy:** how scattered the *meanings* of repeated answers are; high = likely hallucination.
- **Day-One Test:** the first-2-hours script proving Witness can read Claude's chat and type into its message box.
- **Multi-armed bandit / Thompson sampling:** a simple form of reinforcement learning that tries several options, tracks how often each succeeds, and increasingly picks the best one while still exploring.
- **Experience memory:** stored confirmed cases, retrieved by similarity and shown to the judge model as examples.
- **Confirmed signal:** evidence-backed verdict, user feedback, verified fix, or labeled dataset example — the only things Witness learns from.
- **AX / Accessibility API:** macOS API that exposes on-screen UI elements and text to assistive apps.